

LAPORAN PRATIKUM PEKAN 6
STRUKTUR DATA

Disusun Oleh:

Rifqi Aditya

2511533002

Dosen Pengampu :

Dr. Wahyudi, S.T, M.T

Asisten Praktikum :

Sherly Sukmadira



DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS
2026

KATA PENGANTAR

Puji syukur ke hadirat Allah SWT karena atas rahmat dan karunia-Nya saya dapat menyelesaikan laporan praktikum mata Struktur Data tentang *Double Linked List*.

Laporan ini disusun untuk memenuhi salah satu tugas praktikum sekaligus sebagai sarana pembelajaran dalam memahami dasar-dasar pemrograman menggunakan bahasa Java. Melalui praktikum ini, penulis dapat mempelajari penggunaan *Double Linked List*.

Penulis menyadari bahwa laporan ini masih jauh dari sempurna. Oleh karena itu, penulis mengucapkan terima kasih kepada dosen pengampu, asisten praktikum, serta rekan-rekan yang telah membantu dalam proses penyusunan laporan ini.

Akhir kata, semoga laporan ini dapat memberikan manfaat, baik bagi penulis sendiri maupun bagi pembaca, khususnya dalam memperdalam pemahaman tentang pemrograman dasar dengan Java.

Padang, 14 Mei 2025

Penulis

Daftar Pustaka

BAB I PENDAHULUAN.....	4
1.1 Latar Belakang	4
1.2 Tujuan	4
1.3 Manfaat.....	4
BAB II PEMBAHASAN.....	5
2.1 Double Linked List	5
2.1.1 Kode Program	5
2.1.2 Uraian Kode Program.....	5
2.2 Double Linked List – Penelusuran	6
2.2.1 Kode Program	6
2.2.2 Uraian Kode Program.....	6
2.2.3 Output	7
2.3 Double Linked List – Insert	8
2.3.1 Kode Program	8
2.3.2 Uraian Kode Program.....	9
2.3.3 Output	10
2.4 Double Linked List – Hapus	11
2.4.1 Kode Program	11
2.4.2 Uraian Kode Program.....	12
2.4.3 Output	13
BAB III TUGAS	14
3.1 Main Class.....	14
3.1.1 Kode Program	14
3.1.2 Uraian Kode Program.....	14
3.2 Driver Class.....	15
3.2.1 Kode Program	15
3.2.2 Uraian Kode Program.....	16
3.3 Output.....	17
BAB III KESIMPULAN	19

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam dunia pemrograman, struktur data merupakan fondasi penting untuk mengelola informasi secara efisien di dalam memori komputer. Salah satu struktur data linear yang fleksibel adalah Linked List. Namun, pada Single Linked List, navigasi data hanya dapat dilakukan satu arah, yang sering kali membatasi efisiensi dalam manipulasi data tertentu.

Double Linked List (DLL) hadir sebagai solusi dengan menyediakan dua penunjuk (pointer), yaitu next dan prev, pada setiap node. Hal ini memungkinkan program untuk melakukan penelusuran data baik ke depan maupun ke belakang secara efisien. Implementasi DLL sangat relevan dalam kehidupan sehari-hari, seperti pada sistem browser history atau manajemen playlist musik, di mana pengguna membutuhkan kemampuan untuk berpindah ke urutan selanjutnya maupun kembali ke urutan sebelumnya dengan cepat. Melalui praktikum ini, mahasiswa diharapkan mampu memahami logika penghubungan antar node serta implementasi algoritma tambah dan hapus data pada struktur DLL.

1.2 Tujuan

1. Memahami konsep dasar dan struktur anatomi dari *Double Linked List* yang terdiri dari atribut data, pointer next, dan pointer prev.
2. Mampu mengimplementasikan Double Linked List dalam pemrograman menggunakan pointer.
3. Melatih kemampuan logika pemrograman dalam mengelola alokasi memori secara dinamis.

1.3 Manfaat

1. Meningkatkan pemahaman tentang struktur data, khususnya manajemen pointer dan node.
2. Membantu mahasiswa dalam mengembangkan kemampuan pemrograman yang lebih fleksibel dan efisien.
3. Meningkatkan kemampuan analisis dan pemecahan masalah dalam manipulasi struktur data linier.

BAB II

PEMBAHASAN

2.1 Double Linked List

2.1.1 Kode Program

```
1 package Pekan6_2511533002;
2 public class NodeDLL_2511533002 {
3     // mendefinisikan kelas nde
4     int data_3002;
5     NodeDLL_2511533002 next_3002;
6     NodeDLL_2511533002 prev_3002;
7
8     // konstruktor
9     public NodeDLL_2511533002(int data_3002) {
10         this.data_3002 = data_3002;
11         this.next_3002 = null;
12         this.prev_3002 = null;
13     }
14 }
```

2.1.2 Uraian Kode Program

1. **int data_3002:** Variabel ini berfungsi untuk menyimpan nilai atau data yang bertipe integer.
2. **NodeDLL_2511533002 next_3002:** Pointer atau referensi yang menunjuk ke alamat node berikutnya dalam list.
3. **NodeDLL_2511533002 prev_3002:** Pointer atau referensi yang menunjuk ke alamat node sebelumnya dalam list.
4. **public NodeDLL_2511533002(int data_3002):** Konstruktor ini digunakan untuk menginisialisasi objek node baru saat dibuat.
5. **this.data_3002 = data_3002;** Mengisi atribut data_3002 dengan nilai yang diterima melalui parameter.
6. **this.next_3002 = null;** dan **this.prev_3002 = null;** Secara default, saat sebuah node baru dibuat dan belum dihubungkan ke list, pointer next dan prev diatur bernilai null untuk menghindari kesalahan referensi memori.

2.2 Double Linked List – Penelusuran

2.2.1 Kode Program

```
1 package Pekan6_2511533002;
2 public class PenelusuranDLL_2511533002 {
3     // fungsi penelusuran maju
4     static void forwardTraversal_2511533002(NodeDLL_2511533002 head_3002) {
5         // memulai penelusuran dari head
6         NodeDLL_2511533002 curr_3002 = head_3002;
7         // lanjutkan sampai akhir
8         while (curr_3002 != null) {
9             // print data
10            System.out.print(curr_3002.data_3002 + " <-> ");
11            // pindah ke node berikutnya
12            curr_3002 = curr_3002.next_3002;
13        }
14        //print spasi
15        System.out.println();
16    }
17    //fungsi penelusuran mundur
18    static void backwardTraversal_2511533002(NodeDLL_2511533002 tail_3002) {
19        // memulai penelusuran dari head
20        NodeDLL_2511533002 curr_3002 = tail_3002;
21        // lanjutkan sampai akhir
22        while (curr_3002 != null) {
23            // print data
24            System.out.print(curr_3002.data_3002 + " <-> ");
25            // pindah ke node berikutnya
26            curr_3002 = curr_3002.prev_3002;
27        }
28        //print spasi
29        System.out.println();
30    }
31
32    public static void main(String[] args) {
33        //cetak DLL
34        NodeDLL_2511533002 head_3002 = new NodeDLL_2511533002(1);
35        NodeDLL_2511533002 second_3002 = new NodeDLL_2511533002(2);
36        NodeDLL_2511533002 third_3002 = new NodeDLL_2511533002(3);
37
38        head_3002.next_3002 = second_3002;
39        second_3002.prev_3002 = head_3002;
40        second_3002.next_3002 = third_3002;
41        third_3002.prev_3002 = second_3002;
42
43        System.out.println("Penelusuran maju : ");
44        forwardTraversal_2511533002(head_3002);
45
46        System.out.println("Penelusuran mundur : ");
47        backwardTraversal_2511533002(third_3002);
48    }
49 }
```

2.2.2 Uraian Kode Program

1. **NodeDLL_2511533002 curr_3002 = head_3002;** : Membuat variabel pointer bernama curr_3002 dan meletakkannya di posisi node paling depan head_3002.
2. **while (curr_3002 != null)** : Menjalankan perulangan secara terus-menerus selama penunjuk curr_3002 masih ada isi.
3. **curr_3002 = curr_3002.next_3002;** : Memindahkan penunjuk curr_3002 maju ke node selanjutnya agar perulangan bisa berlanjut.
4. **NodeDLL_2511533002 curr_3002 = tail_3002;** : Membuat variabel penunjuk curr_3002 dan meletakkannya di posisi node paling akhir.

5. **while (curr_3002 != null) {** : Menjalankan perulangan selama penunjuk curr_3002 masih menunjuk pada sebuah node.
6. **curr_3002 = curr_3002.prev_3002;** : Memindahkan penunjuk curr_3002 mundur ke node sebelumnya (prev_3002).
7. **NodeDLL_2511533002 head_3002 = new NodeDLL_2511533002(1);** :
Menciptakan objek node pertama di dalam memori yang berisi data angka 1.
8. **NodeDLL_2511533002 second_3002 = new NodeDLL_2511533002(2);** :
Menciptakan objek node kedua yang berisi data angka 2.
9. **NodeDLL_2511533002 third_3002 = new NodeDLL_2511533002(3);** :
Menciptakan objek node ketiga yang berisi data angka 3.
10. **head_3002.next_3002 = second_3002;** : Menyambungkan ikatan maju dari node pertama menuju ke node kedua.
11. **second_3002.prev_3002 = head_3002;** : Menyambungkan ikatan mundur dari node kedua kembali menuju ke node pertama.
12. **second_3002.next_3002 = third_3002;** : Menyambungkan ikatan maju dari node kedua menuju ke node ketiga.
13. **third_3002.prev_3002 = second_3002;** : Menyambungkan ikatan mundur dari node ketiga kembali menuju ke node kedua.

2.2.3 Output

```
Penelusuran maju :  
1 <-> 2 <-> 3 <->  
Penelusuran mundur :  
3 <-> 2 <-> 1 <->
```

2.3 Double Linked List – Insert

2.3.1 Kode Program

```
1 package Pekan6_2511533002;
2 public class InsertDLL_2511533002 {
3     // menambahkan node di awal DLL
4     static NodeDLL_2511533002 InsertBegin_2511533002(NodeDLL_2511533002 head_3002, int data_3002) {
5         // buat node baru
6         NodeDLL_2511533002 new_node_3002 = new NodeDLL_2511533002(data_3002);
7         // jadikan pointer nextnya head
8         new_node_3002.next_3002 = head_3002;
9         // jadikan pointer prev head ke new node
10        if (head_3002 != null) {
11            head_3002.prev_3002 = new_node_3002;
12        }
13        return new_node_3002;
14    }
15    // fungsi menambah node di akhir
16    public static NodeDLL_2511533002 InsertEnd_2511533002(NodeDLL_2511533002 head_3002, int newData_3002) {
17        // buat node baru
18        NodeDLL_2511533002 newNode_3002 = new NodeDLL_2511533002(newData_3002);
19        // jika DLL null jadikan head
20        if (head_3002 == null) {
21            head_3002 = newNode_3002;
22        }
23        else {
24            NodeDLL_2511533002 curr_3002 = head_3002;
25            while (curr_3002.next_3002 != null) {
26                curr_3002 = curr_3002.next_3002;
27            }
28            curr_3002.next_3002 = newNode_3002;
29            newNode_3002.prev_3002 = curr_3002;
30        }
31        return head_3002;
32    }
33    // fungsi menambah node di posisi tertentu
34    public static NodeDLL_2511533002 InsertAtPosition_2511533002(NodeDLL_2511533002 head_3002, int pos_3002, int new_data_3002) {
35        // buat node baru
36        NodeDLL_2511533002 new_node_3002 = new NodeDLL_2511533002(new_data_3002);
37        if (pos_3002 == 1) {
38            new_node_3002.next_3002 = head_3002;
39            if (head_3002 != null) {
40                head_3002.prev_3002 = new_node_3002;
41            }
42            head_3002 = new_node_3002;
43            return head_3002;
44        }
45        NodeDLL_2511533002 curr_3002 = head_3002;
46        for (int i = 1; i < pos_3002 - 1 && curr_3002 != null; ++i);
47        curr_3002 = curr_3002.next_3002;
48        if (curr_3002 == null) {
49            System.out.print("Posisi tidak ada");
50            return head_3002;
51        }
52        new_node_3002.prev_3002 = curr_3002;
53        new_node_3002.next_3002 = curr_3002.next_3002;
54        curr_3002.next_3002 = new_node_3002;
55        if (new_node_3002.next_3002 != null) {
56            new_node_3002.next_3002.prev_3002 = new_node_3002;
57        }
58        return head_3002;
59    }
60    public static void printList_2511533002(NodeDLL_2511533002 head_3002) {
61        NodeDLL_2511533002 curr_3002 = head_3002;
62        while (curr_3002 != null) {
63            System.out.print(curr_3002.data_3002 + " <-> ");
64            curr_3002 = curr_3002.next_3002;
65        }
66        System.out.println();
67    }
68    public static void main(String[] args) {
69        // membuat DLL 2 <-> 3 <-> 5
70        NodeDLL_2511533002 head_3002 = new NodeDLL_2511533002(2);
71        head_3002.next_3002 = new NodeDLL_2511533002(3);
72        head_3002.next_3002.prev_3002 = head_3002;
73        head_3002.next_3002.next_3002 = new NodeDLL_2511533002(5);
74        head_3002.next_3002.next_3002.prev_3002 = head_3002.next_3002;
```



```

75 // cetak DLL awal
76 System.out.print("DLL awal : ");
77 printList_2511533002(head_3002);
78 // tambah 1 di awal
79 head_3002 = InsertBegin_2511533002(head_3002, 1);
80 System.out.print("Simpul 1 di tambah di awal :");
81 printList_2511533002(head_3002);
82 // tambah 6 di akhir
83 System.out.print("Simpul 6 di tambah di akhir");
84 int data_3002 = 6;
85 head_3002 = InsertEnd_2511533002(head_3002, data_3002);
86 printList_2511533002(head_3002);
87 //menambahkan node 4 di posisi 4
88 System.out.print("Tambah node 4 di posisi 4 : ");
89 int data2_3002 = 4;
90 int pos_3002 = 4;
91 head_3002 = InsertAtPosition_2511533002(head_3002, pos_3002, data2_3002);
92 printList_2511533002(head_3002);
93 }
94 }

```

2.3.2 Uraian Kode Program

1. **static NodeDLL_2511533002 InsertBegin_2511533002(NodeDLL_2511533002 head_3002, int data_3002) {** : Mendeklarasikan fungsi statis untuk menambah node di paling depan. Fungsi ini mengembalikan sebuah Node dan menerima dua parameter: letak head (kepala) saat ini dan data angka yang ingin dimasukkan.
2. **if (head_3002 != null) {** : Memeriksa apakah linked list saat ini tidak kosong
3. **head_3002.prev_3002 = new_node_3002;** : Jika tidak kosong, arahkan penunjuk mundur
4. **public static NodeDLL_2511533002 InsertEnd_2511533002(NodeDLL_2511533002 head_3002, int newData_3002) {** : Mendeklarasikan fungsi untuk menambahkan node di ujung belakang list.
5. **NodeDLL_2511533002 newNode_3002 = new NodeDLL_2511533002(newData_3002);** : Membuat objek node baru dengan data yang diinputkan.
6. **if (head_3002 == null) {** : Mengecek apakah linked list dalam keadaan kosong.
7. **head_3002 = newNode_3002;** : Jika benar-benar kosong, maka node baru ini otomatis akan langsung menjadi head.
8. **while (curr_3002.next_3002 != null) {** : Memulai perulangan untuk menelusuri setiap node sampai menemukan node yang paling terakhir
9. **curr_3002.next_3002 = newNode_3002;** : Setelah sampai di node paling akhir, sambungkan penunjuk next dari node akhir tersebut ke node yang baru.
10. **newNode_3002.prev_3002 = curr_3002;** : Sambungkan penunjuk prev dari node baru kembali ke node akhir sebelumnya. Sekarang node baru resmi berada di ujung.
11. **head_3002, int pos_3002, int new_data_3002) {** : Mendeklarasikan fungsi untuk menyisipkan node di posisi/indeks urutan tertentu.
12. **if (pos_3002 == 1) {** : Memeriksa jika posisi yang diminta oleh pengguna adalah posisi 1
13. **if (head_3002 != null) { head_3002.prev_3002 = new_node_3002; }** : Jika head lama ada isinya, kaitkan prev-nya kembali ke node baru.
14. **for (int i = 1; i < pos_3002 - 1 && curr_3002 != null; ++i);** : Menjalankan perulangan bernilai kosong.

15. **if (curr_3002 == null)** { : Mengecek apakah posisi penelusuran melampaui panjang list yang ada.
16. **if (new_node_3002.next_3002 != null)** { : Mengecek apakah masih ada node lain setelah posisi disisipkan.
17. **new_node_3002.next_3002.prev_3002 = new_node_3002;** : Jika ada, arahkan prev dari node tersebut kembali ke node baru agar rantai dua arah utuh.
18. **NodeDLL_2511533002 head_3002 = new NodeDLL_2511533002(2);** : Membuat inisialisasi node pertama yang bernilai angka 2.
19. **head_3002.next_3002 = new NodeDLL_2511533002(3);** : Menambahkan node kedua yang bernilai angka 3 secara berurutan.
20. **head_3002.next_3002.prev_3002 = head_3002;** : mundur dari node angka 3 kembali ke node 2.
21. **head_3002.next_3002.next_3002 = new NodeDLL_2511533002(5);** : Menambahkan node ketiga bernilai 5 di belakang node 3.
22. **head_3002.next_3002.next_3002.prev_3002 = head_3002.next_3002;** : mundur dari node angka 5 ke node 3
23. **head_3002 = InsertBegin_2511533002(head_3002, 1);** : Menyisipkan node berangka 1 di posisi paling depan.
24. **head_3002 = InsertEnd_2511533002(head_3002, data_3002);** : Memasukkan angka 6 tersebut ke bagian akhir list.
25. **int data_3002 = 6;** : Menyimpan nilai angka 6 ke dalam variabel penampung sementara.
26. **head_3002 = InsertEnd_2511533002(head_3002, data_3002);** : Memasukkan angka 6 tersebut ke bagian akhir list.

2.3.3 Output

```
DLL awal : 2 <-> 3 <-> 5 <->
Simpul 1 di tambah di awal :1 <-> 2 <-> 3 <-> 5 <->
Simpul 6 di tambah di akhir1 <-> 2 <-> 3 <-> 5 <-> 6 <->
Tambah node 4 di posisi 4 : 1 <-> 2 <-> 4 <-> 3 <-> 5 <-> 6 <->
```

2.4 Double Linked List – Hapus

2.4.1 Kode Program

```
1 package Pekan6_2511533002;
2 public class HapusDLL_2511533002 {
3     //fungsi menghapus DLL
4     public static NodeDLL_2511533002 delHead_2511533002(NodeDLL_2511533002 head_3002) {
5         if (head_3002 == null) {
6             return null;
7         }
8         head_3002 = head_3002.next_3002;
9         if (head_3002 != null) {
10             head_3002.prev_3002 = null;
11         }
12         return head_3002;
13     }
14     //fungsi menghapus di akhir
15     public static NodeDLL_2511533002 delLast_2511533002(NodeDLL_2511533002 head_3002) {
16         if (head_3002 == null) {
17             return null;
18         }
19         if (head_3002.next_3002 == null) {
20             return null;
21         }
22         NodeDLL_2511533002 curr_3002 = head_3002;
23         while (curr_3002.next_3002 != null) {
24             curr_3002 = curr_3002.next_3002;
25         }
26         // update pointer previous node
27         if (curr_3002.prev_3002 != null) {
28             curr_3002.prev_3002.next_3002 = null;
29         }
30         return head_3002;
31     }
32     // fungsi menghapus node di posisi tertentu
33     public static NodeDLL_2511533002 delPos_2511533002(NodeDLL_2511533002 head_3002, int pos_3002) {
34         //jika DLL kosong
35         if (head_3002 == null) {
36             return head_3002;
37         }
```

```
38         NodeDLL_2511533002 curr_3002 = head_3002;
39         //telusuri sampai ke node yang akan di hapus
40         for (int i = 1; curr_3002 != null && i < pos_3002; ++i) {
41             curr_3002 = curr_3002.next_3002;
42         }
43         //jika posisi tidak ditemukan
44         if (curr_3002 == null) {
45             return head_3002;
46         }
47         //update pointer
48         if (curr_3002.prev_3002 != null) {
49             curr_3002.prev_3002.next_3002 = curr_3002.next_3002;
50         }
51         if (curr_3002.next_3002 != null) {
52             curr_3002.next_3002.prev_3002 = curr_3002.prev_3002;
53         }
54         //jika yang dihapus head
55         if (head_3002 == curr_3002) {
56             head_3002 = curr_3002.next_3002;
57         }
58         return head_3002;
59     }
60     //fungsi mencetak DLL
61     public static void printList_2511533002(NodeDLL_2511533002 head_3002) {
62         NodeDLL_2511533002 curr_3002 = head_3002;
63         while (curr_3002 != null) {
64             System.out.print(curr_3002.data_3002 + " ");
65             curr_3002 = curr_3002.next_3002;
66         }
67         System.out.println();
68     }
69     public static void main(String[] args) {
70         //buat sebuah DLL
71         NodeDLL_2511533002 head_3002 = new NodeDLL_2511533002(1);
72         head_3002.next_3002 = new NodeDLL_2511533002(2);
73         head_3002.next_3002.prev_3002 = head_3002;
```

```

75     head_3002.next_3002.next_3002 = new NodeDLL_2511533002(3);
76     head_3002.next_3002.next_3002.prev_3002 = head_3002.next_3002;
77
78     head_3002.next_3002.next_3002.next_3002 = new NodeDLL_2511533002(4);
79     head_3002.next_3002.next_3002.next_3002.prev_3002 = head_3002.next_3002.next_3002;
80
81     head_3002.next_3002.next_3002.next_3002.next_3002 = new NodeDLL_2511533002(5);
82     head_3002.next_3002.next_3002.next_3002.next_3002.prev_3002 = head_3002.next_3002.next_3002.next_3002;
83
84     System.out.print("DLL awal : ");
85     printList_2511533002(head_3002);
86
87     System.out.print("Setelah head di hapus : ");
88     head_3002 = delHead_2511533002(head_3002);
89     printList_2511533002(head_3002);
90
91     System.out.print("Setelah node terakhir di hapus : ");
92     head_3002 = dellast_2511533002(head_3002);
93     printList_2511533002(head_3002);
94
95     System.out.print("Menghapus node ke 2 : ");
96     head_3002 = delPos_2511533002(head_3002, 2);
97
98     printList_2511533002(head_3002);
99     }
100 }

```

2.4.2 Uraian Kode Program

1. **public static NodeDLL_2511533002 delHead_2511533002(NodeDLL_2511533002 head_3002)** { : Membuat fungsi statis untuk menghapus node pertama (*head*), yang akan mengembalikan *head* baru setelah proses penghapusan.
2. **if (head_3002 == null) { return null; }** : Jika *list* kosong, langsung hentikan proses dan kembalikan null.
3. **if (head_3002.next_3002 == null) { return null; }** : Jika *list* hanya berisi satu node tunggal, maka setelah dihapus *list* akan kosong, sehingga kembalikan null.
4. **while (curr_3002.next_3002 != null)** { : Menjalankan perulangan selama penunjuk belum menabrak node yang paling terakhir.
5. **if (curr_3002.prev_3002 != null)** { : Memastikan bahwa node sebelum node terakhir tersebut benar-benar ada (bukan
6. **curr_3002.prev_3002.next_3002 = null;** : Mengakses node sebelum node terakhir, lalu memutus next menjadi null.
7. **for (int i = 1; curr_3002 != null && i < pos_3002; ++i)** { : Memulai perulangan untuk menelusuri *list* maju satu per satu, sampai mencapai titik sebelum posisi yang diinginkan atau *list* habis.
8. **if (curr_3002 == null) { return head_3002; }** : Jika posisi target ternyata melebihi total panjang *list*, batalkan penghapusan dan kembalikan head.
9. **if (curr_3002.prev_3002 != null)** { : Memeriksa apakah node yang akan dihapus ini memiliki node sebelum dirinya.
10. **curr_3002.prev_3002.next_3002 = curr_3002.next_3002;** : Memutus ikatan next dari node sebelumnya, lalu menyambungkannya melompati node saat ini, langsung menuju node setelahnya.
11. **curr_3002.next_3002.prev_3002 = curr_3002.prev_3002;** : Memutus ikatan prev dari node setelahnya, lalu menyambungkannya mundur melompati node saat ini, langsung menuju node sebelumnya.
12. **if (head_3002 == curr_3002)** { : Kondisi khusus: memeriksa apakah node yang dihapus ternyata adalah node nomor satu

13. **head_3002 = curr_3002.next_3002;** : Jika iya, maka pindahkan head ke node
sebelahnya agar head tidak ikut hilang
14. **NodeDLL_2511533002 head_3002 = new NodeDLL_2511533002(1);** : Membuat
node pertama (sebagai head) yang berisi angka 1.
15. **head_3002.next_3002 = new NodeDLL_2511533002(2);** : Menambahkan node
kedua yang berisi angka 2.
16. **head_3002.next_3002.prev_3002 = head_3002;** : Mengaitkan arah mundur dari
node 2 kembali ke node 1.
17. **head_3002.next_3002.next_3002 = new NodeDLL_2511533002(3);** :
Menambahkan node ketiga berisi angka 3.
18. **head_3002.next_3002.next_3002.prev_3002 = head_3002.next_3002;** : Mengaitkan
arah mundur dari node 3 ke node 2.
19. **head_3002.next_3002.next_3002.next_3002 = new NodeDLL_2511533002(4);** :
Menambahkan node keempat berisi angka 4.
20. **head_3002.next_3002.next_3002.next_3002.prev_3002 =**
head_3002.next_3002.next_3002; : Mengaitkan arah mundur dari node 4 ke node 3.
21. **head_3002.next_3002.next_3002.next_3002.next_3002 = new**
NodeDLL_2511533002(5); : Menambahkan node kelima berisi angka 5.
22. **head_3002.next_3002.next_3002.next_3002.next_3002.prev_3002 =**
head_3002.next_3002.next_3002.next_3002; : Mengaitkan arah mundur dari node 5
ke node 4.
23. **head_3002 = delHead_2511533002(head_3002);** : Mengeksekusi perintah hapus
depan.
24. **head_3002 = delLast_2511533002(head_3002);** : Mengeksekusi perintah hapus
akhir.
25. **head_3002 = delPos_2511533002(head_3002, 2);** : Mengeksekusi perintah hapus
node pada indeks ke-2.

2.4.3 Output

```
DLL awal : 1 2 3 4 5
Setelah head di hapus : 2 3 4 5
Setelah node terakhir di hapus : 2 3 4
Menghapus node ke 2 : 2 4
```

BAB III

TUGAS

3.1 Main Class

3.1.1 Kode Program

```
1 package Pekan6_2511533002;
2
3 public class Lagu_2511533002 {
4     private String judul_3002;
5     private String penyanyi_3002;
6
7     public Lagu_2511533002 next_3002;
8     public Lagu_2511533002 prev_3002;
9
10    public Lagu_2511533002(String judul_3002, String penyanyi_3002) {
11        this.judul_3002 = judul_3002;
12        this.penyanyi_3002 = penyanyi_3002;
13        this.next_3002 = null;
14        this.prev_3002 = null;
15    }
16
17    public String getJudul_3002() {
18        return judul_3002;
19    }
20
21    public String getPenyanyi_3002() {
22        return penyanyi_3002;
23    }
24
25    public void setJudul_3002(String judul_3002) {
26        this.judul_3002 = judul_3002;
27    }
28
29    public void setPenyanyi_3002(String penyanyi_3002) {
30        this.penyanyi_3002 = penyanyi_3002;
31    }
32 }
```

3.1.2 Uraian Kode Program

1. `this.judul_3002 = judul_3002;` : Memasukkan data judul yang kita inputkan tadi ke dalam variabel `judul_3002` milik objek tersebut.
2. `this.penyanyi_3002 = penyanyi_3002;` : Memasukkan data penyanyi yang kita inputkan ke dalam variabel `penyanyi_3002`.
3. `this.next_3002 = null;` : Secara bawaan, saat lagu baru dibuat, penunjuk lagu selanjutnya masih dibiarkan kosong.
4. `this.prev_3002 = null;` : Secara bawaan, penunjuk lagu sebelumnya juga dibiarkan kosong.

3.2 Driver Class

3.2.1 Kode Program

```
39 public void tampilMaju_3002() {
40     if (head_3002 == null) {
41         System.out.println("Playlist kosong!");
42         return;
43     }
44
45     Lagu_2511533002 current = head_3002;
46     System.out.println("=== Daftar Playlist (Maju) ===");
47     while (current != null) {
48         System.out.println("Judul: " + current.getJudul_3002() + " - Penyanyi: " + current.getPenyanyi_3002());
49         current = current.next_3002;
50     }
51 }
52
53 public void tampilMundur_3002() {
54     if (tail_3002 == null) {
55         System.out.println("Playlist kosong!");
56         return;
57     }
58
59     Lagu_2511533002 current = tail_3002;
60     System.out.println("=== Daftar Playlist (Mundur) ===");
61     while (current != null) {
62         System.out.println("Judul: " + current.getJudul_3002() + " - Penyanyi: " + current.getPenyanyi_3002());
63         current = current.prev_3002;
64     }
65 }
66
67 public void carilagu_3002(String judul_3002) {
68     if (head_3002 == null) {
69         System.out.println("Playlist kosong! Tidak dapat mencari lagu.");
70         return;
71     }
72
73     Lagu_2511533002 current = head_3002;
74     boolean ditemukan = false;
75
76     while (current != null) {
77         if (current.getJudul_3002().equalsIgnoreCase(judul_3002)) {
78             System.out.println("Lagu ditemukan: " + current.getJudul_3002() + " oleh " + current.getPenyanyi_3002());
79             ditemukan = true;
80             break;
81         }
82         current = current.next_3002;
83     }
84
85     if (!ditemukan) {
86         System.out.println("Lagu dengan judul '" + judul_3002 + "' tidak ditemukan di playlist.");
87     }
88 }
89
90 public static void main(String[] args) {
91     LaguDriver_2511533002 playlist = new LaguDriver_2511533002();
92     Scanner scanner = new Scanner(System.in);
93     int pilihan = 0;
94
95     while (pilihan != 6) {
96         System.out.println("\n=== Playlist Musik ===");
97         System.out.println("1. Tambah Lagu");
98         System.out.println("2. Hapus Lagu Pertama");
99         System.out.println("3. Lihat Playlist (Maju)");
100        System.out.println("4. Lihat Playlist (Mundur)");
101        System.out.println("5. Cari Lagu");
102        System.out.println("6. Keluar");
103        System.out.print("Pilihan: ");
104
105        if (scanner.hasNextInt()) {
106            pilihan = scanner.nextInt();
107            scanner.nextLine();
108        }
109
110        switch (pilihan) {
111            case 1:
112                System.out.print("Judul: ");
113                String judulInput = scanner.nextLine();
```

```

113         System.out.print("Penyanyi: ");
114         String penyanyiInput = scanner.nextLine();
115         playlist.tambahLagu_3002(judulInput, penyanyiInput);
116         break;
117     case 2:
118         playlist.hapusLaguAwal_3002();
119         break;
120     case 3:
121         playlist.tampilMaju_3002();
122         break;
123     case 4:
124         playlist.tampilMundur_3002();
125         break;
126     case 5:
127         System.out.print("Masukkan judul lagu yang dicari: ");
128         String cariJudul = scanner.nextLine();
129         playlist.carilagu_3002(cariJudul);
130         break;
131     case 6:
132         System.out.println("Keluar dari program. Terima kasih!");
133         break;
134     default:
135         System.out.println("Pilihan tidak valid! Silakan coba lagi.");
136     }
137 } else {
138     System.out.println("Input tidak valid! Harap masukkan angka.");
139     scanner.nextLine();
140 }
141 }
142 scanner.close();
143 }
144 }

```

3.2.2 Uraian Kode Program

1. **public void tambahLagu_3002(String judul_3002, String penyanyi_3002) { :** Membuat fungsi untuk menambahkan lagu baru ke bagian akhir playlist..
2. **Lagu_2511533002 nodeBaru = new Lagu_2511533002(judul_3002, penyanyi_3002); :** Menciptakan objek lagu baru berdasarkan parameter yang dimasukkan.
3. **if (head_3002 == null) { :** Mengecek apakah playlist saat ini masih dalam keadaan kosong.
4. **head_3002 = nodeBaru; tail_3002 = nodeBaru; :** Jika benar kosong, maka lagu baru ini otomatis akan menjadi lagu pertama sekaligus lagu terakhir.
5. **tail_3002.next_3002 = nodeBaru; :** Sambungkan penunjuk maju dari lagu yang saat ini berada di akhir, menuju ke lagu yang baru ditambahkan.
6. **nodeBaru.prev_3002 = tail_3002; :** Sambungkan penunjuk mundur dari lagu baru kembali ke lagu akhir yang lama.
7. **tail_3002 = nodeBaru; :** Perbarui posisi tail agar menunjuk ke lagu yang baru tersebut.
8. **if (head_3002 == null) { :** Jika playlist kosong, cetak peringatan dan langsung keluar dari fungsi tanpa melakukan apa-apa.
9. **if (head_3002 == tail_3002) { :** Jika playlist hanya memiliki persis satu buah lagu (posisi awal dan akhir sama).
10. **head_3002 = null; tail_3002 = null; :** Kosongkan nilai head dan tail sekaligus agar playlist kembali kosong sepenuhnya.
11. **head_3002 = head_3002.next_3002; :** Pindahkan posisi head ke lagu urutan kedua.
12. **head_3002.prev_3002 = null; :** Putus ikatan mundur dari lagu kedua tersebut agar lagu pertama benar-benar terbangun.
13. **if (head_3002 == null) { :** Cegah penelusuran jika daftar lagu kosong.

14. **Lagu_2511533002 current = head_3002;** : Buat penunjuk current dan mulai letakkan di lagu urutan pertama.
15. **while (current != null) {** : Lakukan perulangan selama current masih menunjuk pada sebuah lagu.
16. **current.getPenyanyi_3002());** : Cetak teks judul dan penyanyi dari lagu yang sedang ditunjuk, menggunakan Getter.
17. **current = current.next_3002;** : Geser current maju ke lagu selanjutnya.
18. **Lagu_2511533002 current = tail_3002;** : Mulai penunjuk dari lagu posisi terakhir.
19. **Lagu_2511533002 current = head_3002;** : Mulai pencarian dari lagu paling pertama.
20. **boolean ditemukan = false;** : Membuat penanda dengan status awal false.
21. **while (current != null) {** : Telusuri setiap lagu satu per satu.
22. **if (current.getJudul_3002().equalsIgnoreCase(judul_3002)) {** : Jika judul lagu yang sedang ditunjuk sama dengan judul yang dicari.
23. **ditemukan = true; break;** : Ubah penanda menjadi true dan hentikan perulangan agar tidak membuang waktu menelusuri lagu sisa.
24. **int pilihan = 0;** : Menyiapkan wadah angka untuk menyimpan pilihan menu pengguna.
25. **while (pilihan != 6) {** : Mulai perulangan menu utama secara tak terbatas selama pengguna tidak mengetik angka 6.

3.3 Output

```
=== Playlist Musik ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 1
Judul: Somebody's Pleasure
Penyanyi: Aziz Hedra
Lagu berhasil ditambahkan!

=== Playlist Musik ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 1
Judul: Beauty And A Beat
Penyanyi: Justin Bieber
Lagu berhasil ditambahkan!
```

```
=== Playlist Musik ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 3
=== Daftar Playlist (Maju) ===
Judul: Somebody's Pleasure - Penyanyi: Aziz Hedra
Judul: Beauty And A Beat - Penyanyi: Justin Bieber

=== Playlist Musik ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 4
=== Daftar Playlist (Mundur) ===
Judul: Beauty And A Beat - Penyanyi: Justin Bieber
Judul: Somebody's Pleasure - Penyanyi: Aziz Hedra

=== Playlist Musik ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 5
Masukkan judul lagu yang dicari: Somebody's Pleasure
Lagu ditemukan: Somebody's Pleasure oleh Aziz Hedra
```

```
=== Playlist Musik ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 2
Lagu pertama berhasil dihapus!

=== Playlist Musik ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 6
Keluar dari program. Terima kasih!
```

BAB III

KESIMPULAN

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa *Doubly Linked List* (DLL) adalah struktur data linier yang menawarkan fleksibilitas tingkat tinggi melalui sistem navigasi dua arah. Berbeda dengan *Singly Linked List* yang pergerakannya terbatas, setiap simpul (*node*) dalam DLL tidak hanya menyimpan data, tetapi juga dilengkapi dengan dua penunjuk referensi: *next* untuk mengarah ke *node* selanjutnya dan *prev* untuk kembali ke *node* sebelumnya. Pemahaman terhadap kerangka dasar yang diapit oleh penunjuk *head* (kepala) dan *tail* (ekor) ini sangat krusial, karena memungkinkan penelusuran data secara efisien dari awal hingga akhir (*forward traversal*) maupun dari akhir kembali ke awal (*backward traversal*).

Selain itu, praktikum ini juga menyoroti manajemen *pointer* sebagai kunci keberhasilan dalam operasi manipulasi data pada DLL. Proses penambahan (*insertion*) dan penghapusan (*deletion*) data—baik di titik awal, di ujung akhir, maupun disisipkan pada posisi tertentu—menuntut ketelitian ekstra dalam logika pemrograman. Kita harus memastikan bahwa kedua ikatan arah (*next* dan *prev*) dari *node* yang bersangkutan diputus dan disambungkan kembali secara utuh agar rantai data tidak *error* atau terputus secara tidak sengaja. Meskipun implementasinya menuntut logika yang sedikit lebih rumit dan alokasi memori tambahan, simulasi aplikasi seperti pengelola daftar putar (*playlist*) musik membuktikan bahwa DLL adalah solusi fundamental yang sangat handal untuk membangun fitur-fitur perangkat lunak yang membutuhkan perpindahan data dua arah secara instan.